

ETF Trade Optimization Code Design 中文说明

1. 这个脚本要解决什么问题

optimize_etf_trade.py 的目标，是用给定的 Dealer Inventory 自动生成一篮子 ETF create 或 redemption 所需的债券交易。

脚本从 holdings 文件开始。这个文件里包含当前组合数据、Bmk Weight、Dealer Inventory 和 Issued Amount。脚本会自动选择一篮子债券，并尽量做到：

- 交易金额接近指定的 PNU notional amount。
- `value gap` 保持在配置好的 hard limit 之内。
- 交易篮子的 duration 尽量接近由 `Bmk Weight` 算出的 benchmark-weighted duration target。
- 全局 duration gap 和各个 duration bucket 的 duration gap 尽量在配置限制内。
- 每个 sector-duration combination 都有覆盖。
- 每个 sector-duration combination 至少选到配置要求的 security 数量。
- 满足 `Dealer Inventory`、redemption 时的当前持仓、`Issued Amount` cap，以及 1,000-share round lot。
- 在更重要的约束满足之后，尽量减少使用的 security 数量。

这段代码没有使用 Gurobi、CPLEX、scipy 或 cvxpy 这类第三方数学优化器。它使用的是一个 deterministic greedy/search heuristic。这个逻辑更像把 trader 手工挑篮子的流程自动化：先定义 target，找到 eligible inventory，建立一个分散化的初始篮子，然后每次按一个 1,000-share round lot 去修正篮子。

2. High-Level Flow

主流程由 main() 控制。简化后的调用关系如下：

```
main()
-> read_holdings()
-> benchmark_weighted_duration()
-> duration_bucket_targets()
-> sector_duration_targets()
-> build_candidates()
-> optimize_trade_with_duration_constraints()
-> portfolio_stats()
-> post_trade_portfolio_duration()
-> summarize_by_duration_bucket()
-> summarize_by_sector_duration_combo()
-> write_csv()
```

main() 更像一个调度器。它负责读取输入、计算 benchmark target、建立 eligible trading universe、调用 optimizer、计算结果统计，并写出输出文件。

3. Configuration 和 Column Mapping

脚本最上方有一个 configuration section。这个部分故意和后面的计算逻辑分开，是为了让脚本迁移到办公室环境时更容易维护。

最重要的对象是 COLUMN_MAPPING。

左边是脚本内部使用的 internal variable name。右边是 input file 里的真实列名。

例如：

```
COLUMN_MAPPING = {  
  "shares": "Shares",  
  "market_value": "Market Value",  
  "duration": "Duration",  
  "dealer_inventory": "Dealer Inventory",  
  "bmk_weight": "Bmk Weight",  
  "issued_amount": "Issued Amount",  
  "sector": "Sector",  
}
```

如果办公室文件使用不同的列名，维护者通常只需要修改这个 mapping 的右边。

代码的其他部分会继续使用稳定的内部名字，比如 shares、duration 或 dealer_inventory。这样可以避免把办公室文件里的列名散落在整个脚本里。

4. 读取并标准化 Holdings

read_holdings(path) 负责读取 input CSV，并给每一行增加内部计算字段。

正式读取数据前，它会调用 validate_required_columns()，确认 mapping 后的 required columns 都存在。如果缺少必要列，脚本会尽早报错，并提示用户更新 COLUMN_MAPPING。

数字字段通过 parse_number() 转换。这个函数可以处理类似 1,234,000.00 的 spreadsheet-style string，也会把空值转换成 0.0。

对每一行有效数据，read_holdings() 会计算：

- `\_shares`
- `\_market\_value`
- `\_duration`
- `\_dealer\_inventory`
- `\_bmk\_weight`
- `\_issued\_amount`
- `\_price\_per\_share`
- `\_sector\_group`
- `\_duration\_bucket`
- `\_bucket\_key`

_price_per_share 的计算方式是：

Market Value / Shares

`_sector_group` 由 `sector_group()` 计算。脚本会把 sector 映射成三组：

- ``federal``
- ``gov``，也就是 provincial 和 municipal 的合并组
- ``corporate``，也就是其他所有 sector 的默认组

`_duration_bucket` 由 `duration_bucket()` 计算。当前配置的 buckets 是：

- ``0-5``
- ``5-10``
- ``10-14``
- ``14-30``

`_bucket_key` 把 sector group 和 duration bucket 合并起来。例子包括：

- ``federal|0-5``
- ``gov|5-10``
- ``corporate|14-30``

optimizer 后续使用的是这套标准化后的数据结构，而不是直接处理原始 spreadsheet string。

5. Benchmark Targets

optimizer 使用 Bmk Weight 来定义 target portfolio structure。

全局 benchmark duration 由 `benchmark_weighted_duration()` 计算：

$$\text{sum}(\text{Bmk Weight} * \text{Duration}) / \text{sum}(\text{Bmk Weight})$$

另外两个函数负责建立 target value 和 target duration：

- ``duration_bucket_targets()``
- ``sector_duration_targets()``

`duration_bucket_targets()` 计算每个 duration bucket 应该分配多少 trade value，以及该 bucket 的 benchmark duration target。

`sector_duration_targets()` 在更细的 sector-duration 层级做同样的事情。例如，它会计算这些 combination 的 target value 和 target duration：

- ``corporate|0-5``
- ``corporate|5-10``
- ``federal|10-14``
- ``gov|14-30``

例如，如果某个 sector-duration combination 占总 Bmk Weight 的 20%，那么指定 PNU trade amount 的 20% 就会分配给这个 combination。

这个设计意味着 target basket 是由 benchmark weight 驱动的，而不是由当前 portfolio market value weight 驱动的。

6. Eligible Trading Candidates

`build_candidates()` 负责建立 optimizer 可以使用的 security 列表。

对于 create trades：

`maximum trade shares <= Dealer Inventory`

对于 redemption trades：

`maximum trade shares <= Dealer Inventory`
`maximum trade shares <= Current Shares`

redemption 里的 current-shares cap 用来确保 post-trade shares 不会变成负数。

create 和 redemption 还都要满足 Issued Amount cap：

`abs(trade shares) <= Issued Amount * 1000 * 0.5`

每个 maximum share amount 都会向下取整到 1,000-share round lot。

每个 candidate 包含：

- 原始 enriched row。
- 最大 round lot 数量。
- 一个 round lot 对应的 market value。
- Duration。
- Sector-duration combination。

这个 candidate list 就是 optimizer 后续使用的 tradable universe。

7. Main Optimizer

当前 production optimizer 是 `optimize_trade_with_duration_constraints()`。

它的工作流程是：

1. 把 candidates 按 sector-duration combination 分组。
2. 检查每个 required combination 是否有足够的 eligible securities。
3. 对每个 sector-duration combination 做 local optimization。
4. 把所有 local baskets 合并成一个 total trade basket。

5. 修正 duration gaps。

6. 用 priority score 修正 value gap、duration gap 和 minimum-security violations。

旧函数 `optimize_trade()` 和 `seed_bucket_coverage()` 还保留在文件里，作为 legacy helpers。但当前的 production path 是 `optimize_trade_with_duration_constraints()`。

8. Combo-Level Optimization

每个 sector-duration combination 由 `optimize_duration_bucket()` 优化。

虽然函数名里有 bucket，但在当前 workflow 里，它通常是针对一个 sector-duration combination 调用的，例如：

```
corporate|5-10
```

这个函数会尽量选择至少要求数量的 securities，目前通常是两个，同时匹配：

- 该 combination 的 target value。
- 该 combination 的 target duration。
- 每个 security 的 maximum lot capacity。
- 1,000-share round lots。

它会搜索一只或两只 securities 的组合。对于两只 securities，它会先估计什么比例最接近 target duration，然后在附近搜索 round-lot counts。

这个 local search 的作用，是让初始篮子在 sector-duration 层级上已经尽量接近 benchmark，之后再进入 global repair stage。

9. 合并 Local Baskets

每个 sector-duration combination 优化完成后，`merge_trades()` 会把所有 local baskets 合并成一个 total trade basket。

内部 trade basket 保存的是 absolute share quantities：

```
{
  row_id: {
    "row": row,
    "shares_abs": absolute_trade_shares
  }
}
```

正负号在最后输出时再加：

- Create trades 输出正的 trade shares。
- Redemption trades 输出负的 trade shares。

10. Repair Passes

初始 basket 建好之后，optimizer 会做两轮 repair passes。

Repair Pass 1: Duration Repair

第一轮 repair pass 主要关注 duration。

如果某个 duration bucket 的 gap 超过配置好的 tolerance，optimizer 会在这个 bucket 内每次增加一个 round lot。

如果 bucket-level duration 已经可以接受，但 global duration 仍然超出 tolerance，optimizer 可以从完整 candidate list 里继续增加。

这里用到的 helper functions 是：

- ``score_after_add_in_duration_bucket()``
- ``score_after_add()``

这些函数问的是一个很直接的问题：

If we add one more 1,000-share lot of this security, does the basket improve?

Repair Pass 2: Hard-Constraint Repair

第二轮 repair pass 可以增加或减少一个 round lot。

它使用 `constraint_score()` 判断一个 trial basket 是否比当前 basket 更好。

只有在不违反某个 sector-duration combination 的 minimum-security requirement 时，才允许减少。

只有当某只 security 仍然有未使用的 capacity 时，才允许增加。这个 capacity 同时受到 Dealer Inventory、redemption 时的 current shares 和 Issued Amount cap 约束。

11. Constraint Priority

`constraint_score()` 是整个文件里最重要的函数之一。

它返回一个 tuple。Python 会从左到右比较 tuple，所以这个 tuple 就变成了业务优先级顺序。

优先级顺序是：

1. 超过 hard CAD limit 的 value gap violation。
2. 超过 configured global limit 的 global duration gap violation。
3. 超过 configured bucket limit 的 duration-bucket gap violation。
4. sector-duration combination 的 minimum-security violation。
5. Absolute value gap。
6. Absolute global duration gap。
7. Maximum duration-bucket gap。

8. Number of selected securities.

这意味着脚本会先尝试满足 hard constraints，然后提升 basket quality，最后才尝试减少 securities 数量。

当前业务约束包括：

- 如果可行，`value gap` 必须在 CAD 300 以内。
- `global duration gap` 应该在 0.1 以内。
- 每个 duration bucket gap 应该在 0.2 以内。
- 每个 sector-duration combination 至少要有配置好的 minimum number of securities。
- `Trade Shares` 必须是 1,000-share round lots。
- Trade size 不能超过 `Dealer Inventory`。
- Redemption trade size 不能超过 current shares。
- Trade size 不能超过 `Issued Amount \* 1000 \* 50%`。

如果所有约束无法同时满足，optimizer 仍然会返回它找到的 best basket。output summary 会明确显示哪些 constraints passed，哪些 failed。

12. Measurement 和 Summary Functions

有些函数不做决策，只负责测量 basket，让 optimizer 和 output reports 可以评估结果。

portfolio_stats() 计算：

- Total trade market value。
- Trade weighted duration。

post_trade_portfolio_duration() 计算应用 create 或 redemption trade 之后的 portfolio duration。

duration_bucket_gaps() 计算每个 duration bucket 的 actual duration minus target duration。

sector_duration_gaps() 计算每个 sector-duration combination 的 actual duration minus target duration。

combo_security_counts() 统计每个 sector-duration combination 里选了多少 securities。

输出 summary 的函数是：

- `summarize\_by\_duration\_bucket()`
- `summarize\_by\_sector\_duration\_combo()`

它们会生成 CSV reports，展示不同 bucket 或 combination 的 value、duration、target duration 和 duration gap。

13. Output Files

脚本会写出 trade list 和 summary files。

trade list 会展示被选中的 securities、trade shares、trade market value、Dealer Inventory、current shares 和 post-trade shares。

summary files 会展示 basket 是否在不同层级上接近 benchmark targets：

- Full trade basket。
- Duration bucket。
- Sector-duration combination。

JSON summary 包含：

- Benchmark duration。
- Trade duration。
- Post-trade portfolio duration。
- Trade value。
- Target value。
- Value gap。
- Global duration gap。
- Bucket duration gaps。
- Sector-duration combination gaps。
- Constraint pass or fail status。

14. Plain-English Summary

这个脚本把手工选择 ETF basket 的过程自动化。

首先，它用 benchmark weight 定义 trade basket 在 duration 和 sector 上应该长什么样。然后，它用 Dealer Inventory、redemption 时的 current holdings、Issued Amount caps 和 round-lot rules，把 universe 过滤成真正可以交易的 bonds。

接着，它建立一个 starting basket，确保每个 sector-duration combination 都被覆盖。每个 combination 会先做 local optimization，所以 basket 不只是分散化，也会尽量接近该部分 portfolio 的 benchmark target。

最后，脚本会每次按一个 1,000-share lot 修正合并后的 basket。它优先处理 hard value gap，然后处理 duration constraints，再处理 coverage rules，最后才尝试减少 securities 数量。

简而言之，这个设计不是黑箱数学优化器。它是一个透明、可维护、rule-based optimizer，遵循 trader 手工挑篮子时会用的同一套逻辑，只是把这套逻辑稳定、自动地执行出来。